HDS06 - Designing and Querying a Clinical Trial Participant Registry

Westbrook University Hospitals NHS Trust, Clinical Trial Participant Registry

Malik J. Lainsbury | github

2026-07-10

Table of contents

Introduction

This report designs a fictional relational database (DB) for the Westbrook University Hospitals NHS Trust Clinical Trial Participant Registry. It is a Microsoft SQL Server (T-SQL), populated with fictional data. The DB is then used to answer seven fictional clinical questions via distinct SQL queries.

> **Note**
>
> **Reproducing the results.** Please run `script/HDS_DB_LAINSBURY_2026.sql`. This is the requested single self-contained script that creates the database, tables, data, trigger and queries in order. It ran without errors from start to finish on my machine. Each query returns one result grid, screenshotted below. I used VS Code with the mssql extension on a local SQL Server.

Part 1: Database Design

Entity Relationship Diagram

My database schema (dbo.) has eight tables. I added 3 additional tables from the five generated from the .csv files: `TrialSite` resolves the trial site many-to-many, `Enrollment` records a participant's participation in a trial at a site and `ConsentSigning` is the append-only audit log of consent events. The ERD is shown in crow's-foot notation with the logic between the connecton in Figure 1.

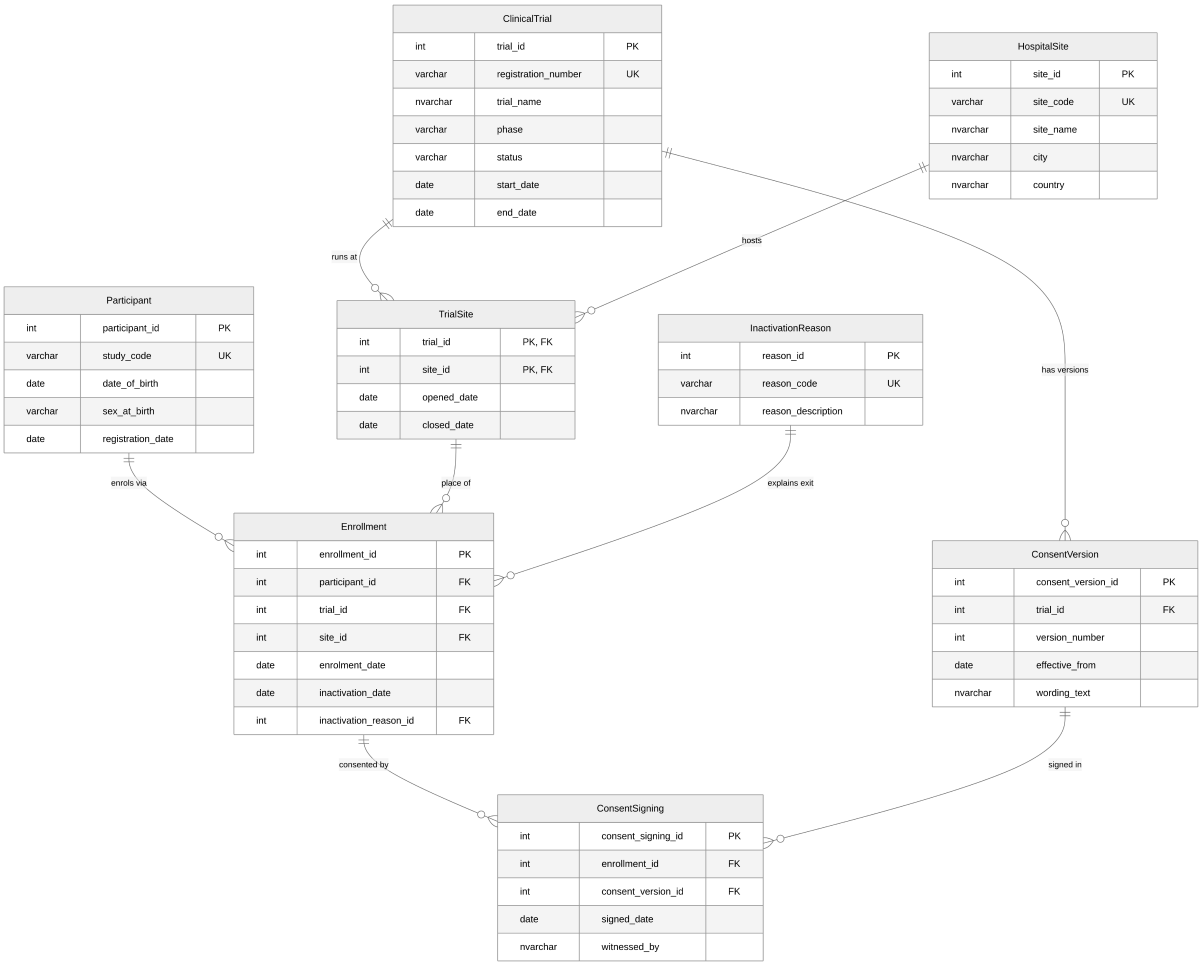

Figure 1: Entity Relationship Diagram (crow's-foot notation).

Table Design Choices

Foreign and Unique Key Design Choices

Every table uses a surrogate `INT IDENTITY` primary key for internal joins, plus one or more **UNIQUE keys (or business keys)** for the values clinical staff would actually recognise.

Every foreign key targets the surrogate key, never the business key. This render the intake slighlty more verbose but is a personal choice for data integrity and maintainability. This study uses externally issued code such as the ISRCTN number. If they were to change, they can then be corrected in one parent row without cascading. The unique keys are used in INSERT look-up subqueries. In the real world, the business keys would be known to the data-entry staff and the look-ups would be hidden behind a user interface/form or via a staging table.

Table Descriptions

Table 1: Table descriptions and key design choices

Table	Purpose and key design choices
ClinicalTrial	One row per trial. PK `trial_id` (IDENTITY). UNIQUE `registration_number` (ISRCTN number) is the externally recognised natural key. Children reference `trial_id`, never the ISRCTN number.
HospitalSite	One row per site. PK `site_id`. UNIQUE `site_code` (e.g. `WBK-GEN`) are the operational natural key. Children reference `site_id`, never `site_code`.
Participant	One pseudonymised participant per row, **no names held**. PK `participant_id`. UNIQUE `study_code`, the pseudonym, is the only safe external identifier. `CHECK (date_of_birth < registration_date)`. `Enrollment` references `participant_id`, never the study code.
InactivationReason	Controlled vocabulary of reasons a participant leaves a trial, so exits report consistently across trials. PK `reason_id`. the UNIQUE `reason_code` is the stable report token. `Enrollment` references `reason_id`, so codes can be renamed safely.
TrialSite	**Link table** resolving the trial- site many-to-many. It also records when each site opened/closed for recruitment on that trial. I use a composite PK `(trial_id, site_id)`. The pairing is the row's identity, so no surrogate is used (unlike every other table). FKs to ClinicalTrial and HospitalSite. Both FKs reference the parents' surrogate keys.
ConsentVersion	Every approved version of a trial's consent form, including wording and effective date. Any old consent versions is never overwritten (under the principle of always add rather than overwrite). PK `consent_version_id`. `UNIQUE (trial_id, version_number)`. The FK references `trial_id`, not the ISRCTN number to follow the same design pattern as other tables.

Table	Purpose and key design choices
Enrollment	A participant enrolled in a trial at a specific site. It carries `(trial_id, site_id)` as a composite FK straight to `TrialSite`, so trial **and** site are captured together and a participant can only enrol where the trial actually runs. Exit is recorded without deleting the row. PK `enrollment_id`; FKs to Participant, TrialSite (composite), InactivationReason; `UNIQUE (participant_id, trial_id, site_id)`; paired-null `CHECK`.
ConsentSigning	Append-only log of every consent signing event: when, which version, who witnessed. PK `consent_signing_id`; FKs to Enrollment and ConsentVersion; `UNIQUE (enrollment_id, consent_version_id)`; protected by an append-only trigger. Both FKs reference surrogate keys. `CHECK` restricts `witnessed_by` to name characters (forename + surname).

Column Design Choices

Column constraints

- Phase/status/sex use `CHECK ... IN (...)` to force controlled vocabularies and clean categorical data for reporting. **Caveat:** the `sex_at_birth` field was intepretated as biological sex and not gender. It only have two values captured for this field. This is a design choice to keep the schema simple and **not** a statement on gender identity. In a production DB this would be reviewed to follow best practice and local healthcare inclusivity guidance.
- I use date checks (`end_date >= start_date`, `closed_date >= opened_date`, `inactivation_date >= enrolment_date`) throughout to prevent logically impossible records.
- I added the paired-null check on `Enrollment` guarantees a leaving date never appears without a reason and vice-versa.
- A `CHECK` restricts `witnessed_by` to name characters only (forename + surname). Names like José shall therefore be written as Jose.
- I used `NVARCHAR(MAX)` for consent wording because it can be long and multi-paragraph. The other text fields are sized to fit the expected content.
- A filtered unique index (UX_Enrollment_OneActivePerParticipant) on Enrollment.participant_id WHERE inactivation_date IS NULL enforces the business rule that a participant may be actively enrolled in only one trial at a time as requested in the brief. This allows unlimited historical enrolments while preventing concurrent participation.
- I added a trigger to enforce the append-only rule on ConsentSigning. It rejects any UPDATE or DELETE operation with a clear error message (51000), while leaving INSERT free. This guarantees the rule regardless of which application or user connects, which a permission grant alone would not.

NULL and NOT NULL Decisions

The two tables below cover every column in the schema (IDENTITY primary keys, and `TrialSite`'s composite-PK columns, are NOT NULL by definition and are omitted).

Table 2: Columns allowing NULL values

Table	Column	Rationale
ClinicalTrial	`end_date`	If NULL,the trial is still running. Current trials have no end date *yet*.
TrialSite	`closed_date`	The site can still be open for recruitment on that trial.
Enrollment	`inactivation_date`, `inactivation_reason_id`	This is a "meaningful absence". the participant is still active in the trial. The pair must be NULL (or populated) together therefore the paired-null `CHECK` keeps them in step.

Table 3: Columns strictly NOT NULL

Table	Column	Rationale
ClinicalTrial	`start_date`, `status`, `phase`	A trial cannot exist without these columns populated.
TrialSite; Enrollment; ConsentVersion	`opened_date`; `enrolment_date`; `effective_from`	Every record's timeline must begin somewhere. Unlike their end-date partners above, an absent value would carry no meaning.
Participant	`date_of_birth`, `registration_date`, `sex_at_birth`	Required for eligibility, age and demographic reporting.
ConsentSigning	`signed_date`, `witnessed_by`	A consent event without a date or witness is not a valid consent event.
ConsentVersion	`wording_text`	A consent version is meaningless without its wording.
Enrollment; ConsentSigning; ConsentVersion	FK columns: `participant_id`, `trial_id`, `site_id`; `enrollment_id`, `consent_version_id`; `trial_id`	A child row cannot exist without its parents. Only exception is with `inactivation_reason_id`, the one deliberately nullable FK, paired with inactivation_date.
ClinicalTrial; HospitalSite; Participant; InactivationReason; ConsentVersion	Unique keys: `registration_number`; `site_code`; `study_code`; `reason_code`; `version_number`	These UNIQUE keys are the look-up targets of the INSERT subqueries. A NULL here would break that pattern and the insertion would fail.

Table	Column	Rationale
ClinicalTrial; HospitalSite; InactivationReason	Descriptive fields: `trial_name`; `site_name`, `city`, `country`; `reason_description`	A reference row without its human-readable label is unusable in reports, it is a reporting requirement.

Relationships Between Tables

Table 4: Relationships between tables

Tables	Type / FK	What it represents
ClinicalTrial – HospitalSite	M:N via `TrialSite` (FK `trial_id`, `site_id`)	A trial runs at many sites and a site hosts many trials. The link table also carries the open/close recruitment dates.
ClinicalTrial -> ConsentVersion	1:M (FK `trial_id`)	One trial accumulates many consent-form versions over its life as the protocol changes.
Participant -> Enrollment	1:M (FK `participant_id`)	A participant may have several enrolments over time, only one active at once as defined per brief (filtered index).
TrialSite -> Enrollment	1:M (composite FK `trial_id, site_id`)	Each enrolment happens at exactly one trial-site pairing that the `TrialSite` link table confirms actually exists.
InactivationReason -> Enrollment	1:M optional (FK `inactivation_reason_id`)	A standardised reason classifies many exits; active enrolments reference none.
Enrollment -> ConsentSigning	1:M (FK `enrollment_id`)	One enrolment can have several signing events (original + re-consent after amendment).
ConsentVersion -> ConsentSigning	1:M (FK `consent_version_id`)	Each signing records which version was signed; one version is signed by many participants.

Many-to-many resolution. The only many-to-many relationship (ClinicalTrial - HospitalSite) is resolved with the `TrialSite` junction table, using the composite primary key `(trial_id, site_id)`. The pairing itself is the row's identity.

Part 2: SQL

INSERT statements

Every INSERT that references another table (e.g. Enrollment needs a trial_id) looks the ID up by a human-recognisable business key. There is no use of hard-coded IDENTITY integers anywhere. It made the script more complexe but is a best practice for maintainability. I approached this assignment as an exercise. In real life production, the business keys would be known to the data-entry staff and the look-ups would be hidden behind a user interface/form or via a staging table.

Additionally I have added data into the DB to allow the queries to return meaningful results. The data is fictional and does not represent any real trial, site or participant.

> **Note**
>
> As requested in the brief. I have only shown the first insert per table in this report to illustrate the pattern. All INSERTs are in `script/HDS_DB_LAINSBURY_2026.sql`.

ClinicalTrial

```
INSERT INTO dbo.ClinicalTrial (registration_number, trial_name, phase, status,
 ↪  start_date, end_date)
VALUES ('ISRCTN10234567', N'CARDIOPROTECT: Cardiac Rehabilitation in Post-MI
 ↪  Patients',
        'III', 'active', '2023-03-01', NULL);
```

HospitalSite

```
INSERT INTO dbo.HospitalSite (site_code, site_name, city, country)
VALUES ('WBK-GEN', N'Westbrook General Hospital', N'London', N'England');
```

Participant

```
INSERT INTO dbo.Participant (study_code, date_of_birth, sex_at_birth,
 ↪  registration_date)
VALUES ('WBK-001', '1958-04-12', 'Male', '2021-11-03');
```

InactivationReason

```
INSERT INTO dbo.InactivationReason (reason_code, reason_description)
VALUES ('consent_withdrawn', N'Participant withdrew their consent to take part
 ↪  in the trial');
```

TrialSite (FK look-ups via subquery)

```
INSERT INTO dbo.TrialSite (trial_id, site_id, opened_date, closed_date)
VALUES (
 (SELECT trial_id FROM dbo.ClinicalTrial WHERE
 ↪  registration_number='ISRCTN10234567'),
 (SELECT site_id  FROM dbo.HospitalSite  WHERE site_code='WBK-GEN'),
 '2023-03-01', NULL);
```

ConsentVersion

```
INSERT INTO dbo.ConsentVersion (trial_id, version_number, effective_from,
 ↪  wording_text)
VALUES (
 (SELECT trial_id FROM dbo.ClinicalTrial WHERE
 ↪  registration_number='ISRCTN10234567'),
 1, '2023-03-01',
 N'I agree to take part in the CARDIOPROTECT study. ... I may withdraw at any
 ↪  time without affecting my medical care.');
```

Enrollment (resolves trial_id and site_id directly from business keys)

```
INSERT INTO dbo.Enrollment (participant_id, trial_id, site_id, enrolment_date,
 ↪  inactivation_date, inactivation_reason_id)
VALUES (
 (SELECT participant_id FROM dbo.Participant   WHERE study_code='WBK-001'),
 (SELECT trial_id       FROM dbo.ClinicalTrial WHERE
 ↪  registration_number='ISRCTN10234567'),
 (SELECT site_id        FROM dbo.HospitalSite  WHERE site_code='WBK-GEN'),
 '2023-03-15', NULL, NULL);
```

ConsentSigning (resolves enrolment + version by business keys)

```
INSERT INTO dbo.ConsentSigning (enrollment_id, consent_version_id, signed_date,
 ↪  witnessed_by)
VALUES (
 (SELECT e.enrollment_id FROM dbo.Enrollment e
    WHERE e.participant_id = (SELECT participant_id FROM dbo.Participant
                              WHERE study_code = 'WBK-001')
      AND e.trial_id       = (SELECT trial_id FROM dbo.ClinicalTrial
                              WHERE registration_number = 'ISRCTN10234567')),
 (SELECT cv.consent_version_id FROM dbo.ConsentVersion cv
    WHERE cv.trial_id = (SELECT trial_id FROM dbo.ClinicalTrial
                         WHERE registration_number = 'ISRCTN10234567')
      AND cv.version_number = 1),
 '2023-03-15', N'Amara Okafor');
```

SQL Queries

Query 1: Current consent wording for CARDIOPROTECT

Clinical question. Before a consent discussion, a research nurse needs the current (most recently effective) consent wording for CARDIOPROTECT: version number, effective date, and full wording.

```
SELECT TOP (1) cv.version_number, cv.effective_from, cv.wording_text
FROM   dbo.ConsentVersion cv
JOIN   dbo.ClinicalTrial  ct ON ct.trial_id = cv.trial_id
WHERE  ct.trial_name LIKE 'CARDIOPROTECT%'
ORDER  BY cv.effective_from DESC;
```

Screenshot.

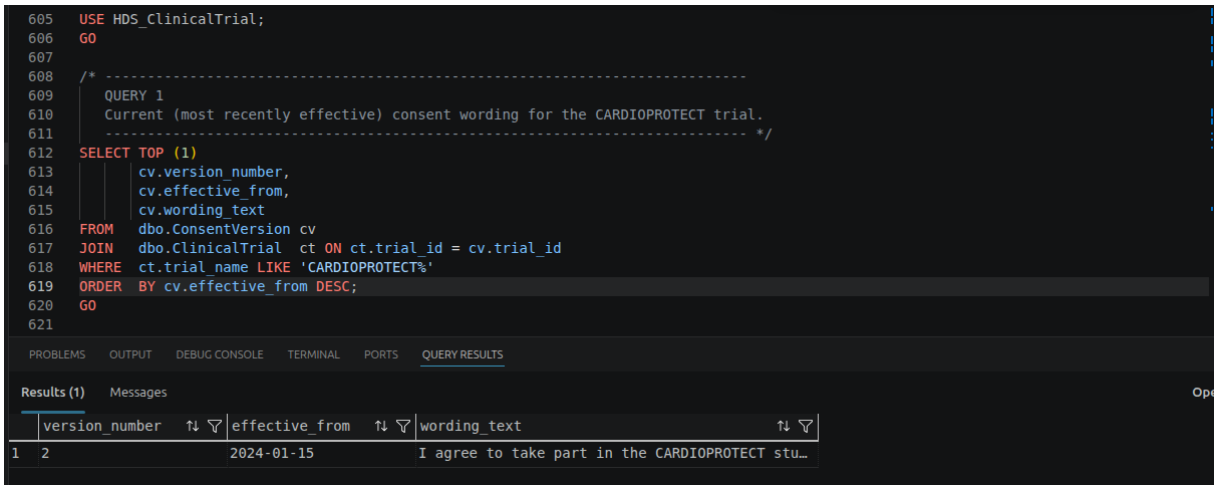

Figure 2: Query 1 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. The query filters to CARDIOPROTECT, orders versions by `effective_from` descending and keeps the top row, so it always returns the latest version even after future amendments.

Data quality: it relies on `effective_from` being accurate, a new version entered with a wrong or missing effective date could show the nurse out-of-date wording.

Query 2: Monthly trial status report

Clinical question. A summary of each trial, its status, how long it has run in days, and whether it has an end date or is ongoing; active trials first, then recruiting, then the rest.

```
SELECT ct.trial_name, ct.status,
       DATEDIFF(DAY, ct.start_date, COALESCE(ct.end_date, CAST(GETDATE() AS
    ↪  DATE))) AS days_running,
       CASE WHEN ct.end_date IS NULL THEN 'Ongoing' ELSE 'Ended' END AS
        ↪  end_date_status
FROM   dbo.ClinicalTrial ct
```

```
ORDER  BY CASE ct.status WHEN 'active' THEN 1 WHEN 'recruiting' THEN 2 ELSE 3
 ↪  END,
          ct.trial_name;
```

Screenshot.

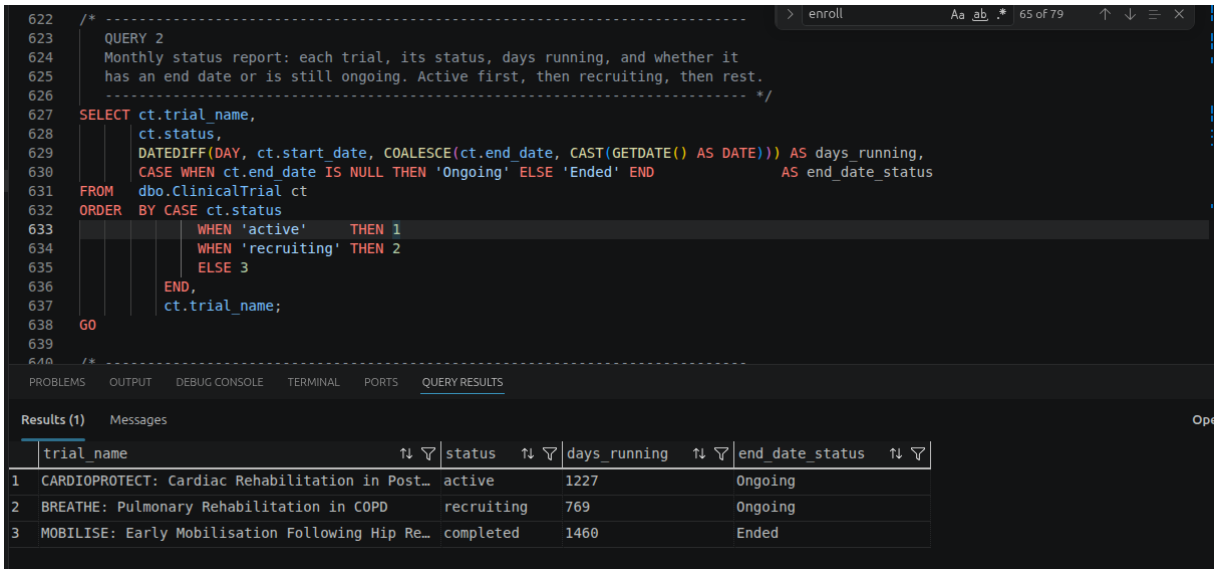

Figure 3: Query 2 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. `COALESCE` substitutes today's date for trials with no end date so "days running" is meaningful for ongoing trials, and a `CASE` expression in `ORDER BY` gives the custom status priority.

Data quality: "days running" for ongoing trials changes every day the report is run, so it is only a dated snapshot.

Query 3: Trial history for participant WBK-003

Clinical question. Full history for WBK-003: every trial, enrolment date, inactivation date, and reason code; ordered by enrolment date. Active enrolments have no reason.

```
SELECT ct.trial_name, e.enrolment_date, e.inactivation_date, ir.reason_code
FROM   dbo.Enrollment       e
JOIN   dbo.Participant      p  ON p.participant_id = e.participant_id
JOIN   dbo.ClinicalTrial    ct ON ct.trial_id      = e.trial_id
LEFT   JOIN dbo.InactivationReason ir ON ir.reason_id =
 ↪  e.inactivation_reason_id
WHERE  p.study_code = 'WBK-003'
ORDER  BY e.enrolment_date;
```

Screenshot.

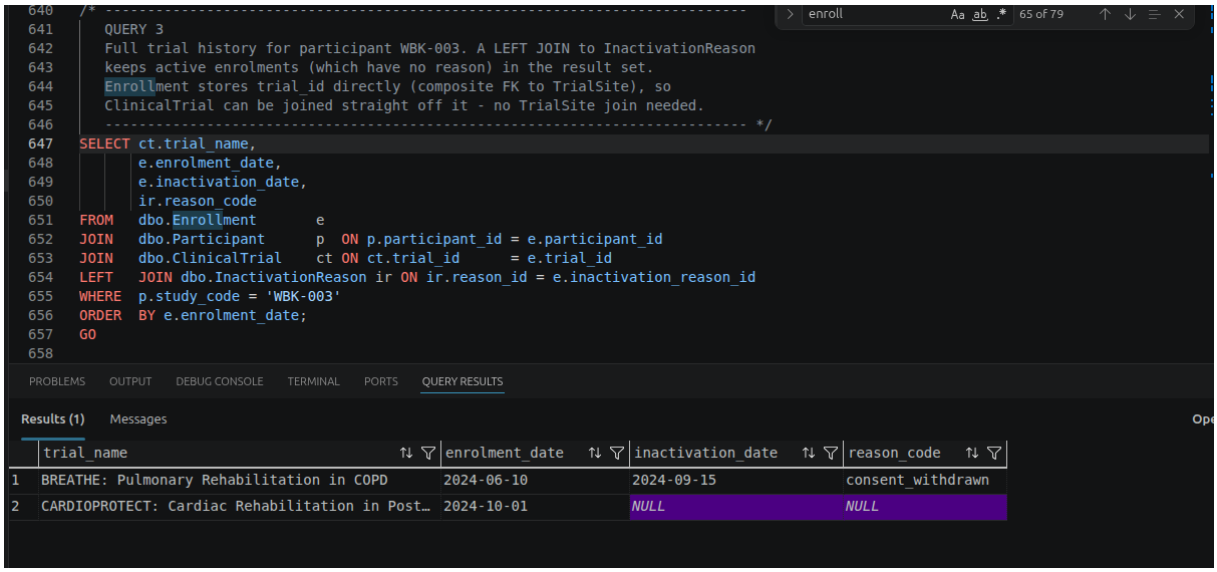

Figure 4: Query 3 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. The `LEFT JOIN` to `InactivationReason` is essential. An active enrolment has a NULL inactivation reason and an INNER join would silently drop it. In my design, `Enrollment` stores `trial_id` directly (as part of its composite FK to `TrialSite`), so `ClinicalTrial` is joined straight off it with no `TrialSite` hop needed.

Data quality: a participant who left but whose `inactivation_reason_id` was never populated would show a leaving date with a blank reason. This is why in the creation of the table, I included the paired-null check constraint to prevents this.

Query 4: Audit of currently active enrolments

Clinical question. Each currently **active** participant with their trial, site, and enrolment date; ordered by trial name then enrolment date.

```
SELECT p.study_code, ct.trial_name, hs.site_name, e.enrolment_date
FROM   dbo.Enrollment    e
JOIN   dbo.Participant   p  ON p.participant_id = e.participant_id
JOIN   dbo.ClinicalTrial ct ON ct.trial_id      = e.trial_id
JOIN   dbo.HospitalSite  hs ON hs.site_id       = e.site_id
WHERE  e.inactivation_date IS NULL
ORDER  BY ct.trial_name, e.enrolment_date;
```

Screenshot.

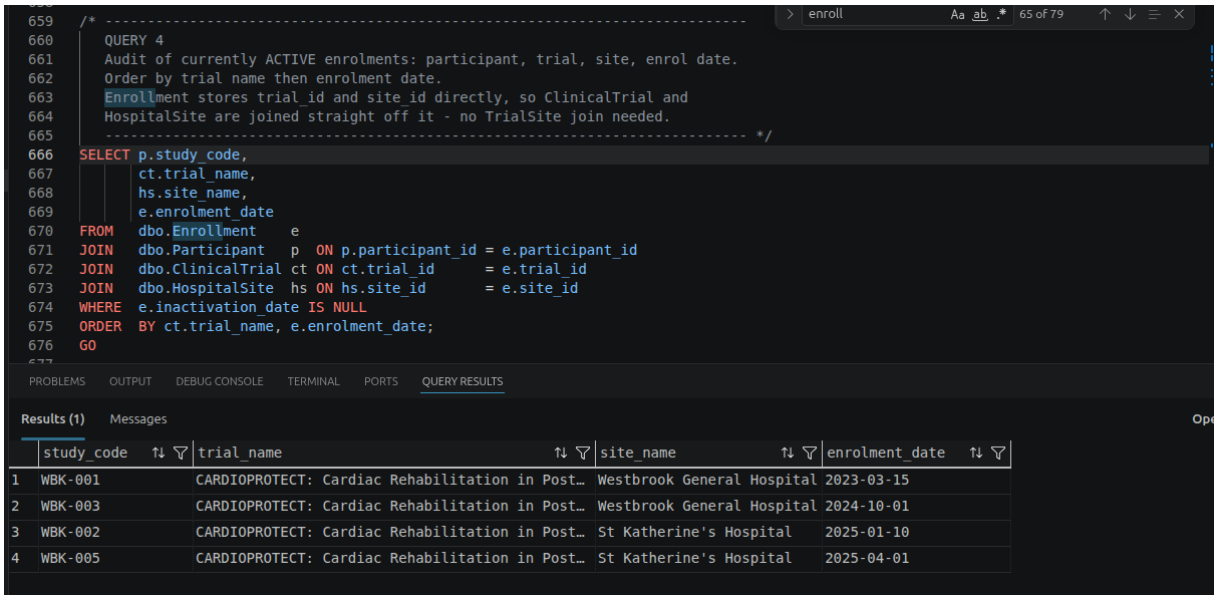

Figure 5: Query 4 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. "Active" is expressed as `inactivation_date IS NULL`. Again, in my table design, `Enrollment` stores `trial_id` and `site_id` directly, so `ClinicalTrial` and `HospitalSite` are joined straight off it, no `TrialSite` hop needed to reconstruct where each active participant sits.

Data quality: the definition of "active" depends on inactivation being recorded promptly. A participant who has left but whose exit was not entered would wrongly appear here.

Query 5: Enrolment count per site (including empty sites)

Clinical question. How many participants have ever been enrolled at each site, including sites with none. Show site name, city, total count, descending.

```
SELECT hs.site_name, hs.city, COUNT(e.enrollment_id) AS total_enrolments
FROM   dbo.HospitalSite hs
LEFT   JOIN dbo.Enrollment e ON e.site_id = hs.site_id
GROUP  BY hs.site_name, hs.city
ORDER  BY total_enrolments DESC;
```

Screenshot.

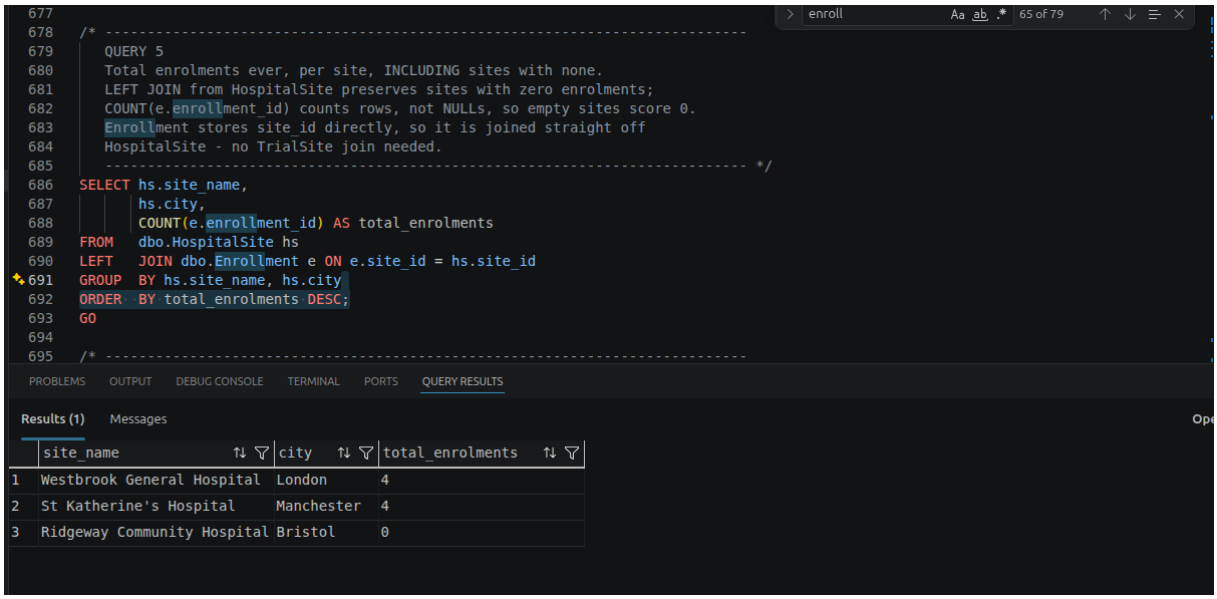

Figure 6: Query 5 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. Starting from `HospitalSite` with a `LEFT JOIN` preserves sites with no enrolments, and `COUNT(e.enrollment_id)` counts rows rather than NULLs so empty sites correctly score 0. `Enrollment` stores `site_id` directly, so it is joined straight off `HospitalSite` with no `TrialSite` hop needed.

Data quality: A site that participates in a trial but has not yet recorded any enrolments is indistinguishable from one that genuinely has none. Same issue as with Query 4 and inactivation, the result depends on the accuracy of the records. Here the zero needs interpretation in context.

Query 6: Trials with an amended protocol

Clinical question. Trials where the protocol was amended (more than one consent version): trial name, current status, number of versions; descending; only where more than one version exists.

```
SELECT ct.trial_name, ct.status, COUNT(cv.consent_version_id) AS version_count
FROM   dbo.ClinicalTrial  ct
JOIN   dbo.ConsentVersion cv ON cv.trial_id = ct.trial_id
GROUP  BY ct.trial_name, ct.status
HAVING COUNT(cv.consent_version_id) > 1
ORDER  BY version_count DESC;
```

Screenshot.

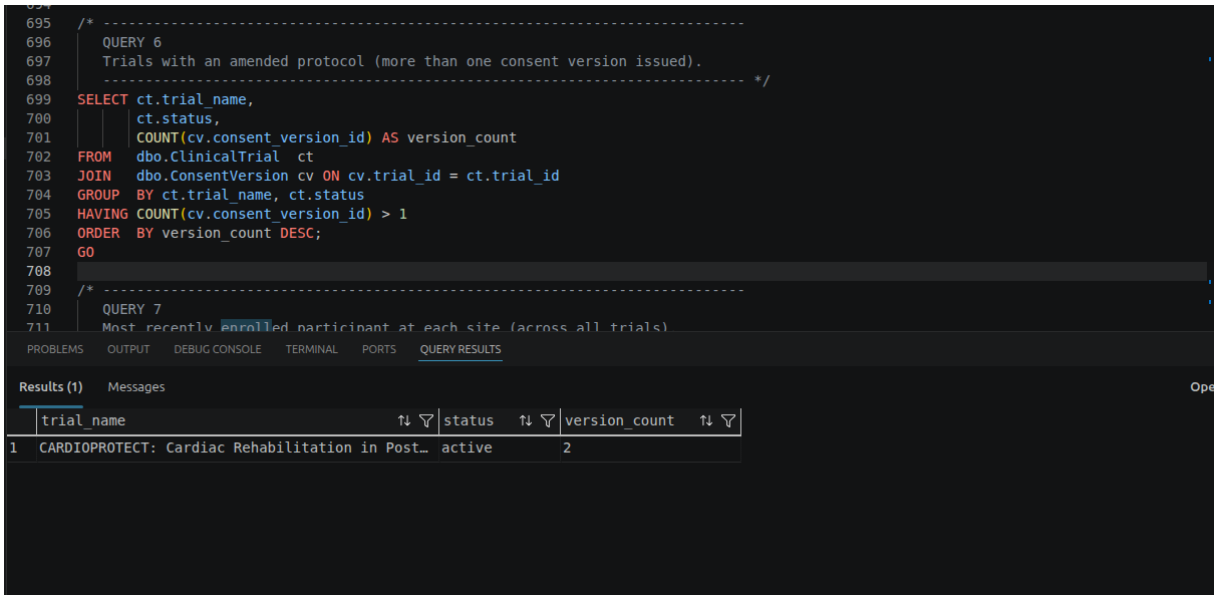

Figure 7: Query 6 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. Grouping by trial and counting versions, then filtering with `HAVING COUNT(...) > 1`, isolates amended trials. I used `HAVING` because it filters the aggregate, which `WHERE` cannot.

Data quality: this counts version rows, so a duplicate version entered in error would overstate the number of genuine amendments.

Query 7: Most recently enrolled participant per site

Clinical question. The most recently enrolled participant at each site across all trials: study code, trial name, site name, enrolment date.

```
WITH RankedEnrolments AS (
    SELECT p.study_code, ct.trial_name, hs.site_name, e.enrolment_date,
           RANK() OVER (PARTITION BY hs.site_id
                        ORDER BY e.enrolment_date DESC) AS rn
    FROM   dbo.Enrollment    e
    JOIN   dbo.Participant   p  ON p.participant_id = e.participant_id
    JOIN   dbo.ClinicalTrial ct ON ct.trial_id      = e.trial_id
    JOIN   dbo.HospitalSite  hs ON hs.site_id       = e.site_id
)
SELECT study_code, trial_name, site_name, enrolment_date
FROM   RankedEnrolments
WHERE  rn = 1
ORDER  BY site_name;
```

Screenshot.

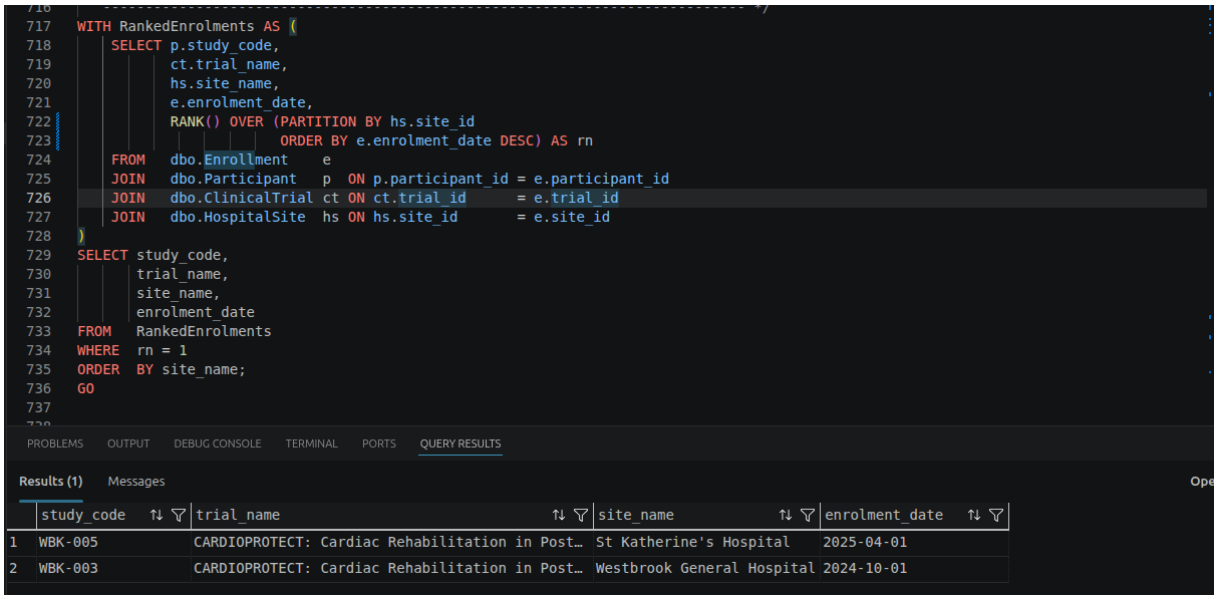

Figure 8: Query 7 output grid (VS Code mssql extension, SQL Server 2025).

Discussion. As seen in class, `RANK()` was used to partitione by site and ordered by enrolment date descending ranks each site's enrolments. Because tied dates share a rank, two participants enrolled at the same site on the same day would both be returned rather than one being picked arbitrarily. `Enrollment` stores `trial_id` and `site_id` directly, so `ClinicalTrial` and `HospitalSite` are joined straight off it with no `TrialSite` hop needed.

Data quality: enrolment is recorded only to the day, so joint "most recent" participants are indistinguishable. A finer timestamp would be needed to single one out such as `DATETIME`.

Part 3: Reflection

Why no participant names?

Storing only a `study_code` pseudonymises the registry, following the data-minimisation principle of GDPR Article 5(1)(c)[1]. Additionally, The Caldicott Principles[2] justify every use of confidential information and use the minimum necessary, reinforcing the same point. A breach of the trial database would therefore not directly expose identities, and names add no information the registry needs to fulfil its research purpose.

Direct identifiers such as names would live in a separate, access-controlled master linkage list held by the research office. In this design, researchers can query and report entirely on `study_code`, so trial analyses can be shared more freely because they carry no personal identifiers. The cost is potential friction and a dependency on that linkage. A broken or out-of-date mapping could mean a participant cannot be reached for a safety issue. Using a pseudonymised design therefore trades day-to-day convenience for a much stronger confidentiality and information-governance position, critical when dealing with personal data.

A situation where the data could mislead

Consider Query 4 and Query 5, which both depend on `inactivation_date`. If a participant has actually left a trial but their exit has not yet been entered, they will still appear as "active" and will be counted in active-enrolment reports — overstating recruitment and potentially prompting unnecessary follow-up contact. The data is not *wrong* field by field; it is *stale*, which is harder to spot. A data manager could detect this by reconciling the registry against source records: cross-checking active enrolments against clinic attendance or against the standardised `InactivationReason` log, and flagging participants with no recent activity (e.g. no consent or visit events for an unusually long period) for review. Preventively, they could add a periodic data-quality report listing long-inactive "active" enrolments, set a service standard for how quickly exits must be recorded, and use the `lost_to_follow_up` reason code consistently so genuine non-contact is captured rather than left as a silent gap. The fix is process plus monitoring, not just a query.

Why consent records are append-only, and how to enforce it

Consent records are the legal and ethical evidence that each participant agreed to the specific protocol version in force **at the time**. They must be append-only because consent is a historical

fact. This requires trial-critical records to carry a complete, tamper-evident audit trail. Editing or deleting a signing event would destroy the evidence that regulators, sponsors and ethics committees rely on to prove valid consent existed. If the rule were broken, a re-consent after a protocol amendment could be silently overwritten or a withdrawn consent erased, making it impossible to demonstrate which wording a participant actually agreed to and undermining every downstream analysis that assumes valid consent.

I enforced this at the database level with an `INSTEAD OF UPDATE, DELETE` trigger on `ConsentSigning` that rejects any such operation with an error, while leaving `INSERT` free (see `script/HDS_DB_LAINSBURY_2026.sql`). This guarantees the rule regardless of which application or user connects, which a permission grant alone would not.

```
CREATE OR ALTER TRIGGER dbo.trg_ConsentSigning_AppendOnly
ON dbo.ConsentSigning
INSTEAD OF UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    THROW 51000, 'ConsentSigning is append-only: rows cannot be updated or
    ↪ deleted.', 1;
    -- throw a custom error to reject the operation so in case of an attempt to
     ↪ update or delete, the trigger will fire and allow the operation to be
     ↪ rejected with a clear message.
END;
```

References

1. Art. 5 GDPR – Principles relating to processing of personal data. *General Data Protection Regulation (GDPR)*.

2. The Caldicott Principles. *GOVUK*.

Appendix A: Data dictionary

A complete column-level reference for every table, derived from the `CREATE TABLE` section of `script/HDS_DB_LAINSBURY_2026.sql`. *Key:* PK = primary key, FK = foreign key, UK = unique (business) key. All surrogate `*_id` keys are `INT IDENTITY(1,1)`.

This dictionary is not static documentation: the metadata section of `script/HDS_DB_LAINSBURY_2026.sql` stores every table description — and the column descriptions where meaning is not self-evident, in particular the meaningful-NULL columns — **in the database catalog itself** as `MS_Description` extended properties. The dictionary can therefore be regenerated at any time (or consumed by data-catalogue tooling) by querying the system views, so the metadata travels with the schema rather than living in a document that can drift out of date:

```
SELECT t.name                                          AS table_name,
       c.name                                          AS column_name,
       TYPE_NAME(c.user_type_id)                       AS data_type,
       IIF(c.is_nullable = 1, 'NULL', 'NOT NULL')      AS nullability,
       CAST(ep.value AS NVARCHAR(400))                 AS column_description
FROM   sys.tables  t
JOIN   sys.columns c ON c.object_id = t.object_id
LEFT   JOIN sys.extended_properties ep
       ON  ep.major_id = c.object_id AND ep.minor_id = c.column_id
       AND ep.class = 1 AND ep.name = 'MS_Description'
ORDER  BY t.name, c.column_id;
```

ClinicalTrial

Table 5: ClinicalTrial

Column	Type	Null	Key	Notes
`trial_id`	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
`registration_number`	VARCHAR(20)	NOT NULL	UK	ISRCTN number; externally recognised natural key.
`trial_name`	NVARCHAR(150)	NOT NULL		Full trial title.
`phase`	VARCHAR(4)	NOT NULL		`CHECK` IN (‘I’,‘II’,‘III’,‘IV’).
`status`	VARCHAR(20)	NOT NULL		`CHECK` IN (‘recruiting’,‘active’,‘completed’,‘suspended’).

Column	Type	Null	Key	Notes
start_date	DATE	NOT NULL		Trial start date.
end_date	DATE	NULL		NULL = ongoing. CHECK (end_date >= start_date).

HospitalSite

Table 6: HospitalSite

Column	Type	Null	Key	Notes
site_id	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
site_code	VARCHAR(10)	NOT NULL	UK	Short operational code, e.g. WBK-GEN.
site_name	NVARCHAR(100)	NOT NULL		Hospital site name.
city	NVARCHAR(60)	NOT NULL		City.
country	NVARCHAR(60)	NOT NULL		Country.

Participant

Table 7: Participant

Column	Type	Null	Key	Notes
participant_id	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
study_code	VARCHAR(15)	NOT NULL	UK	Pseudonym; only external identifier (no names held).
date_of_birth	DATE	NOT NULL		CHECK (date_of_birth < registration_date).
sex_at_birth	VARCHAR(10)	NOT NULL		CHECK IN ('Male','Female'). Records biological sex at birth, not gender identity — only two values are captured for this field.
registration_date	DATE	NOT NULL		Date first registered with the research office.

InactivationReason

Table 8: InactivationReason

Column	Type	Null	Key	Notes
reason_id	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
reason_code	VARCHAR(30)	NOT NULL	UK	Standardised report token, e.g. consent_withdrawn.

Column	Type	Null	Key	Notes
reason_description	NVARCHAR(255)	NOT NULL		Human-readable description of the reason.

TrialSite (link table)

Table 9: TrialSite. The only table in the schema without a surrogate IDENTITY key — the (trial_id, site_id) pairing is itself the row's identity.

Column	Type	Null	Key	Notes
trial_id	INT	NOT NULL	PK, FK	-> ClinicalTrial. Part of the composite PK (trial_id, site_id).
site_id	INT	NOT NULL	PK, FK	-> HospitalSite. Part of the composite PK (trial_id, site_id).
opened_date	DATE	NOT NULL		Date the site opened for recruitment on this trial.
closed_date	DATE	NULL		NULL = still open. CHECK (closed_date >= opened_date).

ConsentVersion

Table 10: ConsentVersion

Column	Type	Null	Key	Notes
consent_version_id	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
trial_id	INT	NOT NULL	FK, UK	-> ClinicalTrial. Part of UNIQUE (trial_id, version_number).
version_number	INT	NOT NULL	UK	CHECK (version_number > 0). Part of the unique key.
effective_from	DATE	NOT NULL		Date this version came into effect.
wording_text	NVARCHAR(MAX)	NOT NULL		Full consent-form wording; never overwritten.

Enrollment

Table 11: Enrollment. A filtered unique index allows only one active enrolment per participant (WHERE inactivation_date IS NULL).

Column	Type	Null	Key	Notes
enrollment_id	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
participant_id	INT	NOT NULL	FK, UK	-> Participant. Part of UNIQUE (participant_id, trial_id, site_id).
trial_id	INT	NOT NULL	FK, UK	Composite FK with site_id -> TrialSite (confirms the site actually runs the trial). Part of the unique key.

Column	Type	Null	Key	Notes
site_id	INT	NOT NULL	FK, UK	Composite FK with trial_id -> TrialSite. Part of the unique key.
enrolment_date	DATE	NOT NULL		Date enrolment began.
inactivation_date	DATE	NULL		NULL = still active. CHECK (inactivation_date >= enrolment_date).
inactivation_reason_id	INT	NULL	FK	-> InactivationReason. Paired-null CHECK with inactivation_date.

ConsentSigning (append-only)

Table 12: ConsentSigning. Protected by an INSTEAD OF UPDATE, DELETE trigger so rows can only ever be inserted.

Column	Type	Null	Key	Notes
consent_signing_id	INT IDENTITY	NOT NULL	PK	Surrogate primary key.
enrollment_id	INT	NOT NULL	FK, UK	-> Enrollment. Part of UNIQUE (enrollment_id, consent_version_id).
consent_version_id	INT	NOT NULL	FK, UK	-> ConsentVersion. Part of the unique key.
signed_date	DATE	NOT NULL		Date the consent was signed.
witnessed_by	NVARCHAR(100)	NOT NULL		Name/role of the witnessing staff member.

Coversheet

MSt Healthcare Data Science — Authentication of practice

I confirm that I have fully read and understood the assignment brief for this module. **Y**

Details

Name	Malik J. Lainsbury
Submission Date	10 – 07 – 26
Word Count: whole assignment including codes	~2,980 word-equivalents (1,531 prose + 7 queries x 150 + 8 INSERT statements x 50)
Word Count: main body excluding abstract, references and supplementary materials	1,531 prose words (appendices excluded as supplementary)

Permission to share your assignment

I do give permission to share my assignment with future MSt participants.

University statement of originality

This assignment is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the Preface and specified in the text. It is not substantially the same as any that I have previously submitted for a degree or diploma or other qualification at the University of Cambridge or any other university or similar institution, or that is being concurrently submitted, except as declared in the Preface and specific in the text. I further state that no substantial part of my Portfolio has already been submitted, nor is being concurrently submitted for any such degree, diploma or other qualification at the University of

Cambridge or any other university or similar institution except as declared in the Preface and specified in the text.

I confirm the statement of originality as above	**Y**

Questions for reflection

1. Which aspects of this assignment are you most uncertain about and/or would most like to receive feedback on? Whether giving `Enrollment` a composite FK `(trial_id, site_id)` straight to `TrialSite` (rather than a single surrogate `trial_site_id` pointer) is considered good practice or over-normalisation; and whether the filtered unique index is an acceptable way to express the "one active trial at a time" rule.

5. Using the wording in the rubric, how would you describe the quality of the different aspects of your work? The ERD, table descriptions and relationships aim at the "high performance" descriptors: all entities identified, the many-to-many resolved with a link table, and each decision justified against the scenario. The script runs end-to-end without errors and every foreign key is resolved by subquery on a business key.

Appendix B: Additional Operational Data

Overview

Assignment section 6.1 specifies: "In addition to the reference data provided, you must also add **your own data** to all the other records you have designed and there should be **enough data of your own to answer all seven queries meaningfully**."

The reference data provided (ClinicalTrial, HospitalSite, InactivationReason, Participant, ConsentVersion) contains no `TrialSite`, `Enrollment`, or `ConsentSigning` rows at all — those three tables are entirely self-authored operational data. Beyond the minimum needed to satisfy the six worked-example requirements, one further **trial transition** was added to enrich Queries 4 and 7, and Ridgeway Community is deliberately left with **zero enrolments** so that Query 5 demonstrably includes a site with none. `TrialSite` uses a composite primary key `(trial_id, site_id)` rather than a surrogate IDENTITY (see Table Descriptions, main body); as a result `Enrollment` stores `trial_id` and `site_id` directly rather than a single `trial_site_id` pointer.

Why a transition, not a second simultaneous enrolment

The schema enforces the brief's rule that a participant may be actively enrolled in only one trial at a time with a filtered unique index:

```
CREATE UNIQUE INDEX UX_Enrollment_OneActivePerParticipant
    ON dbo.Enrollment(participant_id)
    WHERE inactivation_date IS NULL;
```

This permits at most one row per participant with `inactivation_date IS NULL`. Consequently, the addition below **closes** the participant's existing active enrolment first (an `UPDATE` in place, preserving the original row as the brief requires), then **inserts** a new active enrolment — exactly the "left a previous trial and later joined a new one" pattern the scenario itself describes.

Additional Trial Transition Added

WBK-005: BREATHE (Westbrook General) closed, CARDIOPROTECT (St Katherine's) opened

Data added: - `UPDATE`: WBK-005's active BREATHE enrolment at Westbrook General is closed — `inactivation_date = 2025-03-15`, `inactivation_reason_id = clinician_decision` - `INSERT`: new active enrolment — CARDIOPROTECT at St Katherine's, `enrolment_date = 2025-04-01` (the most recent enrolment date in the whole database) - `INSERT`: consent signing — CARDIOPROTECT v2 (the current version), signed 2025-04-01, witnessed by Nurse A. Okafor

Impact on queries: - **Query 7**: gives St Katherine's a genuine choice between two dated candidates (WBK-002's original 2025-01-10 enrolment and WBK-005's 2025-04-01 enrolment), so the `RANK() OVER (PARTITION BY site ...)` logic is doing real selection work rather than returning a trivial single-row partition - **Query 4**: WBK-005 still appears (now under CARDIOPROTECT/St Katherine's instead of BREATHE/Westbrook General)

Why Ridgeway Community Keeps Zero Enrolments

Ridgeway Community is linked to BREATHE in `TrialSite` — it participates in the trial — but no participant is ever enrolled there. This is deliberate: Query 5 asks for enrolment counts *including sites that currently have no enrolments recorded*, and only a genuinely empty site proves the `LEFT JOIN` preserves the zero row (an `INNER JOIN` would silently drop it). It also reflects a real registry situation: a site can be open for recruitment before its first participant arrives.

Data Coverage Summary

Metric	Before addition	After addition
Enrollment rows	7	8
Active enrolments	4	4 (unchanged — the transition closes one, opens one)
Hospital sites with $ $1 enrolment ever	2	2 (Ridgeway deliberately kept at zero for Query 5)
Consent signing events	8	9
Trials each active participant has moved between	0	1 participant (WBK-005) demonstrates the "left one trial, joined another" pattern

Query 4, 5 and 7 results with this data

Query 4 (active enrolments, 4 rows): WBK-001 / CARDIOPROTECT / Westbrook General / 2023-03-15; WBK-003 / CARDIOPROTECT / Westbrook General / 2024-10-01; WBK-002 / CARDIOPROTECT / St Katherine's / 2025-01-10; WBK-005 / CARDIOPROTECT / St Katherine's / 2025-04-01.

Query 5 (enrolments per site, 3 rows): Westbrook General 4; St Katherine's 4; Ridgeway Community 0 — the zero row is preserved by the `LEFT JOIN`.

Query 7 (most recent per site, 2 rows): St Katherine's — WBK-005 / CARDIOPROTECT / 2025-04-01; Westbrook General — WBK-003 / CARDIOPROTECT / 2024-10-01. Ridgeway Community has no enrolments and therefore no row.